Integrating an AI-Driven Multi-Agent System with Simu5G for 5G RAN Energy Optimization (Auto EA-SORAN)

Section 1: Architectural Framework for Integrating AI-driven Agents with Simu5G

This report provides a comprehensive technical guide for integrating a multi-agent, AI-driven control system with the Simu5G network simulator to achieve dynamic energy conservation in 5G Radio Access Networks (RAN). The proposed architecture leverages Simu5G as a high-fidelity digital twin of the network environment, enabling the development, testing, and validation of sophisticated energy-saving policies in a controlled, repeatable, and cost-effective manner. This initial section establishes the foundational co-simulation framework, defining the roles of the external AI agents and the internal simulation components, and detailing the critical communication mechanisms that bridge the two environments.

1.1 The Digital Twin Concept in 5G Network Optimization

The management and control of 5G networks are characterized by immense complexity, driven by features such as network slicing, millimeter-wave frequency bands, and massive MIMO antenna systems.[1] Before deploying new control functionalities, network operators must rigorously test and validate them with precise parameter tuning. High-fidelity system-level simulators provide an indispensable platform for this purpose, enabling rapid assessment and prototyping without requiring access to expensive and operationally critical real-world deployments.[1]

In this context, Simu5G serves as an ideal foundation for creating a digital twin of the 5G RAN.[3] A digital twin is a virtual representation of a physical system that is updated from real-world data and uses simulation to mirror the state and behavior of its physical counterpart. For this project, Simu5G provides a fully controllable virtual environment where network conditions—such as traffic load, user mobility patterns, and inter-cell interference—can be precisely defined and reproduced.[4] This level of control is a significant advantage over testing on live commercial networks, which are subject to unpredictable fluctuations and offer no guarantees of experimental repeatability.[2] By using Simu5G, the

multi-agent system can be trained and evaluated under a wide spectrum of network scenarios, ensuring its robustness and reliability before any potential deployment.

1.2 Proposed Co-Simulation Architecture: A Closed-Loop Control System

The proposed system, comprising four distinct agents (Prediction, Optimization, Feedback, and Orchestration), constitutes a sophisticated closed-loop control architecture. This architecture is designed to continuously monitor the network state, decide on an optimal energy-saving action, validate that action against performance constraints, and finally, actuate the change within the network. Simu5G functions as the "Environment" in this control loop, representing the physical 5G network and providing the necessary feedback to the agents.
The operational flow of this closed-loop system can be conceptualized as follows:

1. **State Observation:** At discrete time intervals, the Simu5G simulation environment generates a comprehensive snapshot of the network's state. This includes critical metrics such as Physical Resource Block (PRB) utilization, the number of active User Equipments (UEs), traffic volume, and current gNodeB (gNB) configurations.
2. **Data Export:** The observed state data is exported from the C++-based Simu5G environment to the external Python environment where the AI agents reside. This data transfer occurs through a defined Inter-Process Communication (IPC) mechanism.
3. **AI Agent Processing:** The exported data is processed sequentially by the AI agents:
    - The **Prediction Agent** (based on the "Farzi" model) ingests the current network state and predicts the future energy consumption under the current configuration.[5]
    - The **Optimization Agent** receives this prediction and formulates an energy conservation plan. This plan may involve actions such as adjusting transmit power, deactivating component carriers, or placing the gNB into a sleep mode.
    - The **Feedback Agent** receives the proposed plan and verifies it against a set of predefined policies and 3GPP-compliant performance thresholds. It ensures that the proposed energy-saving action will not unacceptably degrade the Quality of Service (QoS), such as user throughput or latency. If the plan is too aggressive, it is revised.
4. **Action Import:** The final, validated energy-saving plan, now considered an "action," is transmitted from the Python environment back into the Simu5G simulation.
5. **Actuation:** An in-simulation module, which we will term the **Orchestration Agent Proxy**, receives this action and executes it by dynamically modifying the parameters and state of the target gNodeB models.

This architectural pattern closely mirrors established control system paradigms, such as Model-Predictive Control (MPC), and provides a robust foundation for advanced network automation research. The separation of prediction, optimization, and feedback functions creates a highly intelligent and adaptable control policy that goes beyond simple reactive adjustments.[6] The forward-looking nature of the Prediction Agent, combined with the constraint-aware validation of the Feedback Agent, allows the system to make proactive and safe decisions. This inherent sophistication places stringent requirements on the underlying simulation platform and, most critically, on the communication interface between the simulator and the agents. The simulation must be stable, deterministic, and capable of responding to external commands with low latency to support such a predictive and iterative control framework.

1.3 Interface Mechanisms: Bridging C++ and Python

The efficacy of the closed-loop control system is heavily dependent on the performance and reliability of the communication channel between the OMNeT++ (C++) simulation and the Python-based AI agents. Several IPC mechanisms can be employed, each with distinct trade-offs in terms of implementation complexity, performance, and suitability for real-time control.

- **File-Based Exchange:** This is the simplest method to implement. The Simu5G simulation writes the observed network state to a file (e.g., CSV or JSON) at the end of each time interval. The Python script then reads this file, processes the data, and writes the chosen action to another file, which the simulation reads to actuate the change. While straightforward for initial prototyping and debugging, this approach is characterized by high latency and low throughput due to disk I/O overhead. It is fundamentally unsuitable for control loops that require frequent interaction or timely responses to dynamic network conditions, as would be the case in an MPC-like system.
- **Real-Time Emulation Mode:** Simu5G inherits powerful real-time emulation capabilities from the OMNeT++ and INET frameworks.[8] This mode allows the simulation to run in lockstep with wall-clock time and, more importantly, enables the exchange of IP packets between the simulated network and the host operating system.[2] This feature can be ingeniously repurposed for control signaling. An OrchestrationAgent module within the simulation can be configured to open a TCP or UDP socket and listen for connections from the external Python agents. The Python agents would, in turn, connect to this socket to send action commands and receive state data. This approach offers significantly lower latency and higher throughput than file-based exchange and transforms the simulation into a network-accessible service, which is a clean and highly scalable architectural pattern.
- **Direct Integration with Messaging Libraries (e.g., ZeroMQ, gRPC):** For the most demanding applications requiring the tightest coupling and lowest possible latency, integrating a dedicated messaging library like ZeroMQ is the most robust solution. This

would involve adding the ZeroMQ C++ library to the OMNeT++/Simu5G project build. The OrchestrationAgent in the simulation and the Python agents would both link against their respective ZeroMQ libraries, allowing them to establish a high-performance, asynchronous message-passing bus. This method provides a highly efficient, brokerless communication channel that is ideal for complex control loops involving frequent, low-latency data exchange.

Given the sophisticated, multi-stage nature of the four-agent control policy, a low-latency communication interface is not merely an optimization but a fundamental requirement. A slow, file-based interface would introduce significant delays into the control loop, causing the agents' decisions to be based on stale network state information. This would lead to suboptimal or even unstable control behavior, particularly in a dynamic network environment. Therefore, this report strongly advocates for a socket-based approach, leveraging either Simu5G's built-in real-time emulation mode or a direct integration with a messaging library like ZeroMQ, to ensure the architectural integrity and performance of the proposed system.

Section 2: Modeling 5G Base Station Energy Consumption in Simu5G

A pivotal requirement for this project is a high-fidelity model of gNodeB energy consumption. While Simu5G provides an exceptionally detailed simulation of the 5G NR protocol stack, it does not include a native, built-in energy consumption model for its network nodes.[10] However, this is not an insurmountable limitation. Simu5G is constructed upon the INET Framework, which contains a powerful, generic, and highly extensible suite of modules for energy modeling.[11] This section provides a detailed, step-by-step guide on how to leverage this underlying framework to design, implement, and integrate a custom, state-based energy model for the Simu5G gNodeB.

2.1 The INET Energy Framework: An Untapped Resource

The INET Framework provides a modular and comprehensive system for simulating power consumption, generation, and storage in network nodes.[12] This system is intentionally designed with a clear separation of concerns, which makes it highly flexible and powerful. The core components are defined by C++ interfaces and are intended to be extended to model specific hardware behaviors.[13] The key modules are summarized in Table 2.

Table 2: Core Modules of the INET Energy Framework

Module/Class Name	Type	Core Responsibility	Key Methods/Parameters
IEnergyConsumer	C++ Interface	Defines the contract	getPowerConsumption

		for any module that consumes power. It requires implementing a method to report the current power draw.	()
StateBasedEpEnergyConsumer	Simple Module	A concrete implementation of IEnergyConsumer that models a device with a finite number of discrete power states (e.g., TRANSMITTING, LISTENING, SLEEP).	powerConsumption parameter (a string of power values for each state), changeState() method.
IEnergyStorage	C++ Interface	Defines the contract for any module that stores energy, such as a battery or capacitor.	getResidualEnergyCapacity(), getNominalEnergyCapacity()
SimpleEpEnergyStorage	Simple Module	A concrete implementation of IEnergyStorage that tracks energy capacity in Joules (J). It automatically integrates the power drawn by all connected consumers over time to calculate total energy consumption.	initialEnergyCapacity parameter, nominalCapacity parameter.

The "Ep" suffix in the module names signifies that these models operate on abstract quantities of Energy (in Joules) and Power (in Watts), which is sufficient for system-level energy efficiency analysis.[12] The framework's architecture allows for a single IEnergyStorage module to be connected to multiple IEnergyConsumer modules, a feature that enables the creation of highly detailed and realistic energy models.

2.2 Designing a Custom gNodeBEnergyConsumer Module

To accurately model the energy consumption of a 5G base station, a new custom module, gNodeBEnergyConsumer, will be created. This module will be implemented in C++ and will inherit from INET's StateBasedEpEnergyConsumer, allowing it to leverage the existing state-management and power-integration logic.[12]

2.2.1 Defining Power States

The first step is to define a set of discrete operational states that represent the different power consumption levels of a gNodeB. These states should reflect the impact of both traffic load and explicit energy-saving features, as described in 3GPP standards and related research.[15] A robust set of states would include:

- STATE_ACTIVE_HIGH_LOAD: The gNodeB is fully active and handling a significant traffic load. Power consumption is at its peak.
- STATE_ACTIVE_LOW_LOAD: The gNodeB is active but handling minimal traffic. Power consumption is dominated by the base/static power draw.
- STATE_MICRO_SLEEP: A light sleep mode, corresponding to features like symbol shutdown, where parts of the baseband processing are temporarily deactivated between transmissions. This provides modest energy savings with very low latency for reactivation.
- STATE_DEEP_SLEEP: A deep sleep mode, corresponding to features like RF channel shutdown or component carrier deactivation. This provides substantial energy savings but incurs a higher latency penalty to return to an active state.
- STATE_OFF: The gNodeB is completely powered down (for simulation purposes).

2.2.2 Assigning Power Values

Each of these states will be associated with a specific power consumption value in Watts. A key design principle is to make these values configurable from the omnetpp.ini file, rather than hardcoding them in C++. This allows for easy calibration of the model for different types of base station hardware without recompiling the simulation. The StateBasedEpEnergyConsumer module supports this directly through its powerConsumption parameter, which accepts a space-separated list of power values corresponding to each state.

2.2.3 State Transition Logic

The gNodeBEnergyConsumer module will contain the C++ logic to transition between these states. These transitions will be triggered by control messages received from the in-simulation OrchestrationAgent (as detailed in Section 4). For instance, upon receiving a message with a command to "enter deep sleep," the module will call its internal changeState(STATE_DEEP_SLEEP) method.

2.3 Linking Energy States to Real-Time Simulation Activity

The power consumption of a base station in its active states is not static; it is highly correlated with the traffic load it is serving.[5] A simple, static power value for STATE_ACTIVE_HIGH_LOAD is insufficient for a high-fidelity model. To address this, the power consumption model must be dynamically linked to the real-time activity of the gNodeB's MAC layer scheduler.
A widely accepted and empirically validated approach is to model the total power consumption as a linear function of the radio resource utilization. The gNodeBEnergyConsumer module can be designed to access the statistics recorded by the gNodeB's MAC scheduler at each Transmission Time Interval (TTI). Specifically, it can retrieve the number of PRBs allocated for transmission in the last TTI. The total power consumption (Ptotal) can then be calculated using the following formula:
$$P_{total} = P_{base} + P_{traffic} \times \left(\frac{num_allocated_PRBs}{total_available_PRBs}\right)$$

Where:

- P_{base} is the static power consumption of the gNodeB when idle but active.
- $P_{traffic}$ is the maximum power consumption attributable to traffic-related processing and RF amplification.
- num_allocated_PRBs is the number of PRBs used in the last TTI.
- total_available_PRBs is the total number of PRBs available on the component carrier.

This calculation would be performed within a custom C++ method that overrides the base getPowerConsumption() method. When the module is in an ACTIVE state, it computes this dynamic value; when in a SLEEP or OFF state, it returns the corresponding static power value. This approach directly and realistically ties the energy model to the core data plane activity of the simulation.

2.4 Integrating the Energy Model into the Simu5G gNodeB

The final step is to integrate the custom energy consumer and a standard energy storage module into the Simu5G gNodeB compound module. This is achieved by modifying the gNodeB.ned file to include these new components as submodules:

Code snippet

```
// Inside gNodeB.ned
import inet.power.storage.SimpleEpEnergyStorage;
import project.modules.gNodeBEnergyConsumer; // Path to custom module
```

```
...
compound gNodeB extends eNodeB
{
   //... existing gNodeB submodules...

   submodules:
      energyStorage: SimpleEpEnergyStorage {
         @display("p=100,200;i=device/battery");
      }
      energyConsumer: gNodeBEnergyConsumer {
         @display("p=100,250;i=block/control");
      }
   connections allowunconnected:
      // No connections needed; modules interact via C++ method calls
}
```

Once integrated, the SimpleEpEnergyStorage module will automatically poll the gNodeBEnergyConsumer module at regular intervals to get its current power draw. It then integrates this power over time to maintain an accurate account of the total energy consumed.[14] This energyConsumed statistic, automatically recorded by the storage module, becomes the primary KPI for evaluating the effectiveness of the energy conservation plan. This modular approach enables a far more realistic and detailed energy model than a monolithic one. For instance, a real base station's power is consumed by multiple distinct components, primarily the Radio Unit (RU) and the Baseband Unit (BBU). A more advanced model could feature two separate EnergyConsumer submodules within the gNodeB: one for the RU, whose power draw is a function of PRB utilization and transmit power, and another for the BBU, whose power draw is a function of the number of active UEs and control plane signaling load. Both consumers would draw power from the same SimpleEpEnergyStorage module. This multi-consumer architecture allows the Optimization Agent to develop more nuanced energy-saving plans. For example, it could devise a strategy that reduces BBU processing for low-complexity IoT devices while keeping the RU fully active for high-throughput eMBB users—a level of detail and control that is impossible to achieve with a single, aggregated energy model.

Section 3: Generating Realistic Input Data for the Prediction Agent

The entire control loop is predicated on the ability of the Prediction Agent to make accurate forecasts of energy consumption. The quality of these predictions is, in turn, entirely dependent on the quality and realism of the data used for its training. This section details the

methodology for configuring a Simu5G simulation scenario to produce the specific data streams required by the "Farzi" prediction model and, more broadly, for generating a rich and diverse dataset suitable for training any robust machine learning model for this purpose.

3.1 Deconstructing the Prediction Agent's Input Requirements

An analysis of the "Farzi" GitHub repository and related academic work on base station energy modeling reveals a consistent set of required input features.[5] These features capture the three primary drivers of energy consumption: static configuration, dynamic traffic load, and the state of energy-saving features. Table 1 provides a direct mapping between these required ML features and their corresponding data sources within the Simu5G/INET simulation environment. This table serves as a critical blueprint for the data collection process.

Table 1: Prediction Agent Input Features and Simu5G Data Sources

Prediction Agent Feature	Simu5G/INET Source	Source Type	Configuration/Access Method
PRB Utilization (%)	gNodeB.collector or gNodeB.nic.mac	Signal/Statistic	Enable statistics like avgServedBlocksDl from GNodeBStatsCollector.[19] Access scheduler internals for per-TTI data.
Transmit Power (dBm)	gNodeB.nic.mac	Module Parameter	Set via .ini file: *.gnb.nic.mac.txPower.[21]
Number of Active Carriers	carrierAggregation	Module Parameter	Set via .ini file: *.carrierAggregation.numComponentCarriers.[22]
Bandwidth (MHz)	carrierAggregation.componentCarrier[*]	Module Parameter	Set via .ini file: *.carrierAggregation.componentCarrier.numBands.[23]
Number of Active Users	gNodeB.nic.mac or Binder	C++ State Variable	Query the MAC layer's list of connected UEs or the global Binder module.
Traffic Volume (DL/UL)	ue[*].app[*] or server.app[*]	Signal/Statistic	Record application-level

			throughput statistics (e.g., cbrReceivedThroughput:mean).[22]
Energy-Saving Mode	gNodeB.energyConsumer (custom module)	C++ State Variable	The internal state of the custom energy consumer module (e.g., STATE_DEEP_SLEEP).

3.2 Configuring the Simulation Scenario (omnetpp.ini)

A well-designed omnetpp.ini configuration file is the key to generating meaningful data. It must define not only the static network parameters but also the dynamic elements that create realistic and variable network load.

3.2.1 Network Topology and Static Parameters

The configuration begins by defining the physical layout of the network and the static hardware parameters of the gNodeB, which directly correspond to some of the prediction model's input features.

Ini, TOML

```
# --- omnetpp.ini ---

# Define the network to be simulated
network = Standalone5GNetwork

# gNodeB placement (e.g., center of a 1000x1000m area)
*.gnb.mobility.initialX = 500m
*.gnb.mobility.initialY = 500m

# gNodeB static configuration parameters
*.gnb.nic.mac.txPower = 40dBm  # Corresponds to Transmit Power feature
*.carrierAggregation.numComponentCarriers = 1 # Corresponds to Number of Carriers
*.carrierAggregation.componentCarrier.carrierFrequency = 2GHz
*.carrierAggregation.componentCarrier.numBands = 50 # Corresponds to Bandwidth (50 RBs)
```

3.2.2 UE and Dynamic Traffic Configuration

The most critical aspect of data generation is creating dynamic and variable traffic load, which in turn drives the PRB utilization. A simulation with static UEs running constant bit-rate traffic will produce flat, repetitive data that is of little value for training a machine learning model. Instead, the scenario must incorporate randomness and diversity in both traffic generation and user behavior.
Simu5G, through INET, provides a rich library of application modules and mobility models to achieve this.[20]

Ini, TOML

```
# --- omnetpp.ini (continued) ---

# Number of User Equipments
*.numUe = ${numUes=10, 20, 50, 100} # Use parameter study variable

# UE placement and mobility
*.ue[*].mobility.typename = "RandomWaypointMobility"
*.ue[*].mobility.constraintAreaMinX = 0m
*.ue[*].mobility.constraintAreaMaxX = 1000m
*.ue[*].mobility.constraintAreaMinY = 0m
*.ue[*].mobility.constraintAreaMaxY = 1000m
*.ue[*].mobility.speed = uniform(1mps, 10mps) # Variable UE speed

# Configure diverse applications for UEs
# Example: 50% of UEs run CBR traffic, 50% run VoIP
*.ue[*].numApps = 1
*.ue[0..${numUes/2-1}].app.typename = "CbrUdpCli"
*.ue[0..${numUes/2-1}].app.serverAddress = "server"
*.ue[0..${numUes/2-1}].app.datagramSize = 1000B
*.ue[0..${numUes/2-1}].app.sendInterval = exponential(10ms) # Variable send interval

*.ue[${numUes/2}..*].app.typename = "VoipUdpCli"
*.ue[${numUes/2}..*].app.serverAddress = "server"
```

The configuration above demonstrates several key principles for generating high-quality data. The use of a parameter study variable (${numUes=...}) is particularly important. A single simulation run provides only one trajectory through the state space. To train a model that can

generalize across different traffic conditions, as required by the "Farzi" project objectives [5], it is necessary to run an ensemble of simulations with varying parameters. OMNeT++'s parameter study feature automates this process, allowing the simulation to be run repeatedly with different numbers of UEs, traffic intensities, or mobility patterns.[26] This generates a rich, composite dataset that covers a wide swath of the input feature space, which is essential for preventing model overfitting and ensuring robust generalization.

3.3 Extracting and Recording Simulation Data

Once the simulation is configured to generate dynamic activity, the final step is to record the required data. OMNeT++ provides a powerful signal-based mechanism for statistics recording. Modules can "emit" signals when significant events occur, and these signals can be configured in the .ini file to be recorded as statistics.

- **PRB Utilization:** The most direct way to measure PRB utilization is to access the statistics from the gNodeB's scheduler. The GNodeBStatsCollector module, a submodule of the gNodeB, is designed for this purpose and provides information on resource block utilization.[19] In simplified scenarios, the BackgroundScheduler also emits signals like bgAvgServedBlocksDl that can be recorded.[23]
- **Other Metrics:** Standard INET application modules and Simu5G protocol layers emit a wide range of useful signals for throughput, delay, and packet loss.[3]

The following .ini configuration enables the recording of these statistics:

Ini, TOML

```
# --- omnetpp.ini (continued) ---

# Enable result recording
*.result-recording-modes = +vector, +scalar

# Record gNodeB scheduler statistics (e.g., allocated RBs)
*.gnb.collector.result-recording-modes = +vector
*.gnb.collector.*.scalar-recording = true
*.gnb.collector.*.vector-recording = true

# Record application-level QoS metrics for each UE
*.ue[*].app.**.scalar-recording = true
*.ue[*].app.**.vector-recording = true
```

This configuration will produce two types of output files: scalar files (.sca), which contain summary statistics at the end of the simulation (e.g., mean throughput), and vector files (.vec), which contain time-series data (e.g., PRB utilization recorded at every TTI). The vector files are particularly valuable as they provide the fine-grained, time-series data needed to train a predictive model. These files can then be parsed by an external Python script using standard data analysis libraries to create the final training dataset for the Prediction Agent.

Section 4: Implementing the Multi-Agent Control Loop

With the simulation environment configured and the energy model in place, the next critical phase is to implement the mechanism that allows the external AI agents to exert control over the simulated gNodeBs. This "closes the loop" by translating the energy conservation plan into concrete actions within the simulation. This section details the design of an in-simulation proxy for the orchestration agent and explains the C++ implementation techniques required to dynamically modify the state and parameters of gNodeB modules during a simulation run.

4.1 The In-Simulation Orchestrator Module

To maintain a clean separation of concerns, a new dedicated C++ simple module, the OrchestrationAgent, will be created. This module serves as the in-simulation proxy or endpoint for the external AI-driven orchestration agent. Its primary responsibilities are:

1. **Communication:** To manage the IPC channel (e.g., a TCP socket or ZeroMQ socket) with the external Python environment, receiving action commands.
2. **Command Parsing:** To parse the incoming commands, which will specify the target gNodeB, the action to be performed (e.g., "setTxPower", "enterSleepMode"), and any associated parameters (e.g., a power value of 35 dBm).
3. **Action Execution:** To translate the parsed command into the appropriate function calls or message-passing events within the OMNeT++ simulation to effect the desired change on the target gNodeB module.

This module will be added as a submodule to the top-level network definition, giving it visibility and access to all other modules in the simulation.

4.2 Dynamic Control of gNodeB State

A significant implementation challenge is that Simu5G does not provide a high-level, abstracted API for dynamically reconfiguring gNodeBs mid-simulation (e.g., a function like

gNodeB->setSleepMode(true)).[27] However, this functionality can be achieved by leveraging the fundamental, low-level capabilities of the underlying OMNeT++ simulation kernel. OMNeT++ allows modules to programmatically access and modify the parameters of any other module in the simulation hierarchy.[28]

The OrchestrationAgent can obtain a pointer to the target gNodeB module using its path in the module hierarchy. Once it has this pointer, it can directly manipulate the module's parameters using the par() method. This is a powerful technique for direct control.

The following C++ code snippet illustrates how the OrchestrationAgent could change the transmit power of a gNodeB named "gnb" at runtime:

C++

```
// Inside OrchestrationAgent::handleMessage(cMessage *msg)

// Assuming the command to change power has been received and parsed
double newTxPowerIndBm = 35.0;

// Get a pointer to the simulation's top-level module
cModule *network = getSystemModule();

// Find the target gNodeB module by its path
cModule *gnbModule = network->getSubmodule("gnb");
if (gnbModule == nullptr) {
   EV_ERROR << "gNodeB module not found!";
   return;
}

// Navigate to the MAC submodule where the txPower parameter resides
cModule *macModule = gnbModule->getSubmodule("nic")->getSubmodule("mac");
if (macModule == nullptr) {
   EV_ERROR << "gNodeB MAC module not found!";
   return;
}

// Set the new parameter value
// NOTE: The parameter must be declared as 'volatile' in its NED definition
// to allow for runtime changes.
macModule->par("txPower").setDoubleValue(newTxPowerIndBm);

// It is good practice to re-read parameters in the target module if it
// has a handleParameterChange() method, though not always necessary.
macModule->checkPar();
EV_INFO << "Set gNodeB txPower to " << newTxPowerIndBm << " dBm.";
```

For actions that involve changing the state of our custom gNodeBEnergyConsumer module, direct parameter manipulation is less suitable. A more robust and event-driven approach is to use OMNeT++'s message-passing system.[29] The OrchestrationAgent can create a new cMessage object, perhaps with a specific kind to identify it as a control command, and send it directly to the gNodeBEnergyConsumer submodule of the target gNodeB. The energy consumer's handleMessage() function would then contain the logic to interpret this message and transition to the appropriate internal state (e.g., changeState(STATE_DEEP_SLEEP)). This message-based approach is the standard and preferred pattern for inter-module communication and control in OMNeT++.

4.3 Implementing Energy-Saving Actions

Using the techniques described above, the specific energy-saving actions can be implemented as follows:

- **Adjusting Transmit Power:** This is achieved via direct parameter modification of the txPower parameter in the gNodeB's MAC module, as demonstrated in the code snippet.[21]
- **Activating Sleep Modes:** This is implemented by sending a control message from the OrchestrationAgent to the custom gNodeBEnergyConsumer module. The message's kind or an attached control object would specify the target sleep state (STATE_MICRO_SLEEP or STATE_DEEP_SLEEP). The consumer module then updates its internal state, which immediately alters its reported power draw to the SimpleEpEnergyStorage module.
- **Turning off Component Carriers:** This is a more complex action. It can be simulated by programmatically changing the numComponentCarriers parameter of the global carrierAggregation module.[22] Additionally, the OrchestrationAgent must signal the gNodeB's MAC scheduler to stop allocating PRBs on the deactivated carrier. This prevents UEs from being scheduled on a carrier that is notionally powered down.

It is crucial to recognize that these actuation mechanisms have both direct and indirect consequences within the simulation. Changing the txPower parameter is a direct action. The indirect, systemic effect is how the rest of the network responds to this change. When the transmit power is lowered, subsequent packets sent from that gNodeB will arrive at the UEs with a lower signal strength. The Simu5G physical layer model, which calculates the Signal-to-Interference-plus-Noise Ratio (SINR) for every reception, will factor this in.[31] A lower SINR will likely result in a higher Block Error Rate (BLER), which may trigger more H-ARQ retransmissions at the MAC layer or force the scheduler to select a more robust (and less spectrally efficient) Modulation and Coding Scheme (MCS). These cascading effects are

precisely what the simulation is designed to capture.
This highlights the importance of allowing for a "settling time" within the control loop. After the OrchestrationAgent executes an action, a certain amount of simulation time must be allowed to elapse before the next network state observation is made. This delay ensures that the systemic effects of the action have fully propagated through the network and that the measured KPIs (like throughput and packet loss) reflect the new steady-state behavior. Making control decisions based on transient states immediately following an action can lead to oscillations and instability in the control loop. This settling time is a critical hyperparameter that must be carefully tuned in the external AI agent's control logic.

Section 5: Validating the Energy Conservation Plan: KPIs and Analysis

The final and most critical stage of the project is to quantitatively evaluate the performance of the AI-driven multi-agent system. This involves not only measuring the energy saved but also rigorously assessing the impact of these savings on the network's Quality of Service (QoS) and overall user experience. This section outlines the Key Performance Indicators (KPIs) to be collected, aligns them with 3GPP standards for energy efficiency, and proposes a robust methodology for a comparative analysis to validate the effectiveness of the energy conservation plan. The data collected in this phase provides the crucial input for the Feedback Agent's validation logic.

5.1 Collecting Performance and Energy KPIs

A successful energy conservation strategy is one that minimizes power consumption while adhering to strict performance constraints. Therefore, the validation process must track a balanced set of KPIs covering both energy efficiency and QoS. Table 3 lists the essential KPIs, their sources within the Simu5G/INET environment, and their relevance to the project's goals.

Table 3: Key Performance Indicators for System Validation

KPI	Metric Category	Simu5G Source (Statistic Name)	Relevance to Feedback Agent
Total Energy Consumed (J)	Energy Efficiency	gNodeB.energyStorage.energyConsumed	Primary metric for quantifying the success of the optimization plan.
Average User Throughput (Mbps)	Quality of Service (eMBB)	ue[*].app.endToEndThroughput:mean	Ensures energy-saving actions do not violate

			throughput requirements for enhanced Mobile Broadband services.[10]
End-to-End Latency (ms)	Quality of Service (URLLC)	ue[*].app.endToEndDelay:mean	Critical for validating performance for latency-sensitive applications like industrial automation or remote control.[3]
Packet Loss Rate (%)	Reliability	ue[*].app.packetDropRate:last	Monitors network reliability and ensures that aggressive power-saving does not lead to unacceptable data loss.[3]
MAC TB Loss Rate (%)	Reliability (RAN)	ue[*].nic.mac.macTbLossRate:mean	A lower-layer metric indicating unrecoverable errors at the MAC layer, often a direct consequence of poor SINR.[10]

These KPIs are readily available from the standard statistics collection mechanisms within Simu5G and INET.[14] By enabling scalar and vector recording in the omnetpp.ini file, as described in Section 3, comprehensive logs of these metrics can be generated for post-simulation analysis and for providing real-time data to the Feedback Agent.

5.2 Aligning Simulation KPIs with 3GPP Standards

The user's requirement for the Feedback Agent to use 3GPP standards and policies for verification is a crucial aspect that grounds the simulation work in industry-relevant practices. 3GPP has defined specific KPIs for Energy Efficiency (EE) for 5G networks.[33] The fundamental definition of EE is the ratio of performance to energy consumption:
EE=Energy Consumption (EC)Performance
The definition of "Performance" is context-dependent. For network slices focused on enhanced Mobile Broadband (eMBB), performance is typically measured by the volume of user plane data successfully transmitted.[33] The simulation environment is perfectly suited to calculate this standards-compliant KPI. At the end of a measurement interval, the total number of bytes delivered by the server applications can be summed, and the total energy

consumed can be read from the
energyStorage modules of the gNodeBs. The EE KPI can then be calculated as:
EEeMBB=Total Joules ConsumedTotal Bytes Delivered(Bytes per Joule)
This provides the Feedback Agent with a concrete, standards-aligned metric to evaluate the efficiency of a proposed plan. The agent's policies can be defined as thresholds on this EE value, or more commonly, as constraints on the individual components of the ratio (e.g., "maintain an average user throughput of at least 100 Mbit/s while minimizing energy consumption").

5.3 A Methodology for Comparative Analysis

To demonstrate the tangible benefits of the multi-agent control system, its performance must be compared against a baseline scenario. A rigorous experimental methodology is essential for producing credible and defensible results. The following experimental plan is proposed:

1. **Run the Baseline Scenario:** Configure the simulation with a realistic, dynamic traffic load and user mobility profile, as detailed in Section 3. Run this simulation for a set duration *without* the AI agents active (i.e., the gNodeB operates with a static configuration). Record the total energy consumed by all gNodeBs and the full set of QoS KPIs (throughput, latency, packet loss). This represents the "unoptimized" performance of the network.
2. **Run the Optimized Scenario:** Execute the exact same simulation scenario, using the same random seeds to ensure identical traffic patterns and mobility trajectories. However, in this run, the full four-agent closed-loop control system is enabled. The agents will dynamically adjust the gNodeB configurations throughout the simulation.
3. **Quantify Savings and Impact:** Compare the results from the two runs. The primary result is the percentage of energy saved. However, it is equally important to quantify the trade-offs. The final analysis should present a nuanced conclusion, such as: "The AI-driven agent system achieved a 22% reduction in total network energy consumption. This was accompanied by a 4% decrease in average user throughput and a 7% increase in average end-to-end latency, both of which remained within the predefined QoS thresholds for the eMBB service."

This comparative analysis provides the definitive validation of the project's success. Furthermore, the inherent tension between energy efficiency and other performance KPIs like latency and reliability must be carefully managed.[7] The Feedback Agent is the mechanism designed to navigate this trade-off, but its thresholds should not be monolithic. A single, global latency threshold, for example, is unrealistic in a multi-service 5G network. The latency requirements for a remote surgery application (URLLC) are orders of magnitude stricter than for a video streaming service (eMBB).[34]
To address this, the system should be evolved to be *slice-aware* or *QoS-aware*. The simulation

scenario should be configured with multiple, distinct traffic types representing different services. The recently introduced Service Data Adaptation Protocol (SDAP) layer in 5G is responsible for mapping QoS Flows to Data Radio Bearers, using a QoS Flow Identifier (QFI).[36] By incorporating an SDAP model into the simulation [37], packets for different services can be tagged with unique QFIs. The Feedback Agent's logic can then be made significantly more intelligent, applying different validation thresholds based on a packet's QFI. This allows the Optimization Agent to make more aggressive energy-saving decisions for delay-tolerant eMBB traffic while maintaining a highly conservative posture for latency-critical URLLC traffic. This context-aware approach to feedback and validation is essential for creating an optimization system that is not only effective but also practical and aligned with the diverse service requirements of real-world 5G networks.

Conclusion

This report has detailed a comprehensive architectural and implementation framework for integrating a sophisticated, four-agent AI control system with the Simu5G network simulator. By leveraging Simu5G as a high-fidelity digital twin, the proposed system can be rigorously developed, trained, and validated in a controlled environment, providing a powerful platform for research into energy-efficient 5G RAN management.
The key technical contributions and recommendations outlined are:

1. **A Robust Co-Simulation Architecture:** A closed-loop control architecture has been proposed, defining the interactions between the external Python-based AI agents and the internal C++ simulation. A strong recommendation is made for using a low-latency IPC mechanism, such as Simu5G's real-time emulation mode or a direct ZeroMQ integration, to support the system's advanced, predictive control logic.
2. **A High-Fidelity gNodeB Energy Model:** A step-by-step methodology has been provided for creating a custom gNodeB energy consumption model by extending the powerful but often-overlooked INET Energy Framework. This model links power draw directly to real-time simulation activity, such as PRB utilization, and supports dynamic state changes corresponding to 3GPP energy-saving features.
3. **A Principled Data Generation Strategy:** A guide for configuring Simu5G scenarios to produce realistic and diverse data has been presented. Crucially, this involves moving beyond single simulation runs to conduct automated parameter studies, thereby generating the rich datasets required to train a robust and generalizable Prediction Agent.
4. **A Practical Implementation of Dynamic Control:** The report has detailed the C++ mechanisms for actuating energy-saving plans within the simulation. By leveraging OMNeT++'s fundamental capabilities for dynamic parameter modification and message-passing, the OrchestrationAgent can effectively control gNodeB states, such as transmit power, sleep modes, and carrier activation.
5. **A Standards-Aligned Validation Methodology:** A clear set of energy and QoS KPIs

has been identified for validating the system's performance. By aligning these metrics with 3GPP definitions for Energy Efficiency, the Feedback Agent can make informed, standards-compliant decisions. The proposed comparative analysis provides a clear path to quantifying the tangible benefits of the AI-driven approach.

By following the detailed guidance provided in this report, the described multi-agent system can be successfully integrated with Simu5G. The resulting co-simulation platform will not only serve to validate the initial energy conservation plan but will also provide an extensible and powerful tool for future research into AI-driven network automation, slice-aware resource management, and the complex interplay between energy efficiency and Quality of Service in 5G and beyond.

Works cited

1. A Calibrated and Automated Simulator for Innovations in 5G - arXiv, https://arxiv.org/html/2404.10643v1
2. Scalable Real-Time Emulation of 5G Networks With Simu5G - ResearchGate, https://www.researchgate.net/publication/355690617_Scalable_Real-Time_Emulation_of_5G_Networks_With_Simu5G
3. Detailed Explanation on 5G Simulation : Parameters - Model - Indicators - OMNeT++ TUTORIAL https://omnet-tutorial.com/5g-simulation/
4. Scalable Real-Time Emulation of 5G Networks With Simu5G - SciSpace, https://scispace.com/pdf/scalable-real-time-emulation-of-5g-networks-with-simu5g-2mv2rbff4t.pdf
5. ITU-AI-ML-in-5G-Challenge/5G-Energy-Consumption-Modelling-Solution-Team-Farzi-Data-Scientists - GitHub https://github.com/ITU-AI-ML-in-5G-Challenge/5G-Energy-Consumption-Modelling-Solution-Team-Farzi-Data-Scientists
6. How to Implement 5G Network Simulator Projects (Guide) https://networksimulationtools.com/5g-network-simulator/
7. A Survey on Green Enablers: A Study on the Energy Efficiency of AI-Based 5G Networks https://www.mdpi.com/1424-8220/24/14/4609
8. Overview - Simu5G: Simulator for 5G New Radio Networks, https://simu5g.org/users-guide/overview
9. Running Simu5G as a real-time network emulator https://simu5g.org/users-guide/emulation
10. Simu5G: A System-level Simulator for 5G Networks - SciTePress https://www.scitepress.org/Papers/2020/98264/98264.pdf
11. Simu5G: Simulator for 5G New Radio Networks https://simu5g.org/
12. Modeling Power Consumption — INET 4.5.4 documentation, https://inet.omnetpp.org/docs/users-guide/ch-power.html
13. An Extensible Framework for Simulating Battery Consumption in Wireless Networks - Laura Marie Feeney https://sonri.org/publications/Files/omnet10energy.pdf
14. How to Calculate Network Energy Utilization in omnet++ https://omnet-manual.com/how-to-calculate-network-energy-utilization-in-omn

et/
15. AI-based energy consumption modeling of 5G base stations: an energy efficient approach, https://sussex.figshare.com/articles/conference_contribution/AI-based_energy_consumption_modeling_of_5G_base_stations_an_energy_efficient_approach/26394829
16. Modeling of IoT Devices Energy Consumption in 5G Networks - mediaTUM, https://mediatum.ub.tum.de/doc/1705687/55ahjo0udp2jr6z1159ilkwbr.Modelling_energy_consumption_of_IoT_devices_in_5G_network.pdf
17. Energy Efficiency for 5G and Beyond 5G: Potential, Limitations, and Future Directions - MDPI https://www.mdpi.com/1424-8220/24/22/7402
18. Energy Consumption Modelling for 5G Radio Base Stations with Machine Learning - Lund University Publications https://lup.lub.lu.se/student-papers/record/9133984/file/9133985.pdf
19. BaseStationStatsCollector - Simu5G https://simu5g.org/neddoc/simu5g.corenetwork.statsCollector.BaseStationStatsCollector.html
20. Packages - Simu5G https://simu5g.org/neddoc/packages.html
21. OMNET++ 5G Thesis Ideas - PHD Services - https://phdservices.org/omnet-5g/
22. New Radio Tutorial - Simu5G: Simulator for 5G New Radio Networks, https://simu5g.org/tutorials/NR/doc/index
23. BackgroundScheduler - Simu5G https://simu5g.org/neddoc/simu5g.nodes.backgroundCell.BackgroundScheduler.html
24. Simu5G - 5G NR and LTE/LTE-A user-plane simulation model for OMNeT++ & INET - GitHub https://github.com/Unipisa/Simu5G
25. A Network Simulator-based Method of Dynamic Traffic Generation for 5G Network Slicing - Concordia's Spectrum https://spectrum.library.concordia.ca/id/eprint/990258/1/Ding_MCompSc_S2022.pdf
26. Parameter Studies - OMNeT++ Technical Articles https://docs.omnetpp.org/tutorials/tictoc/part7/
27. gNodeB - Simu5G https://simu5g.org/neddoc/simu5g.nodes.NR.gNodeB.html
28. OMNeT++ - Simulation Manual - Index of - Omnetpp.org 2025, https://doc.omnetpp.org/omnetpp5/manual/
29. OMNeT++ Simulation Library: cSimpleModule Class Reference - Index of, https://doc.omnetpp.org/omnetpp4/api/classcSimpleModule.html
30. Simu5G: A System-level Simulator for 5G Networks - ARPI, accessed on August 31, 2025, https://arpi.unipi.it/retrieve/handle/11568/1039566/609208/SIMULTECH_2020.pdf
31. Simu5G–An OMNeT++ Library for End-to-End Performance Evaluation of 5G Networks - ARPI https://arpi.unipi.it/bitstream/11568/1053174/2/2020%20Access.pdf
32. Steps to Implement Network Energy Management in OMNeT++ https://omnet-manual.com/how-to-implement-network-energy-management-in

-omnet/
33. Energy Efficiency (EE) SA5 work and results - 3GPP https://www.3gpp.org/technologies/energy-efficiency-ee-sa5-work-and-results
34. 5G and Energy Efficiency https://standards-tracker.5g-ppp.eu/sites/default/files/BookletA4_EnergyEfficiency.pdf
35. (PDF) A comparison analysis of 5G key performance indicators based on entropy, https://www.researchgate.net/publication/386292921_A_comparison_analysis_of_5G_key_performance_indicators_based_on_entropy
36. Main modules of the Simu5G model library. | Download Scientific Diagram - ResearchGate https://www.researchgate.net/figure/Main-modules-of-the-Simu5G-model-library_fig3_344434106
37. SDAP-based QoS Flow Multiplexing Support in Simu5G for 5G NR Simulation This publication has emanated from research conducted with the financial support of Science Foundation Ireland under Grant number 13/RC/2077 P2. For the purpose of Open Access, the author has applied a CC-BY public copyright licence to any Author Accepted Manuscript version arising - arXiv https://arxiv.org/html/2508.12785v1